

DEBUG ARENA

Estratégias de Debugging

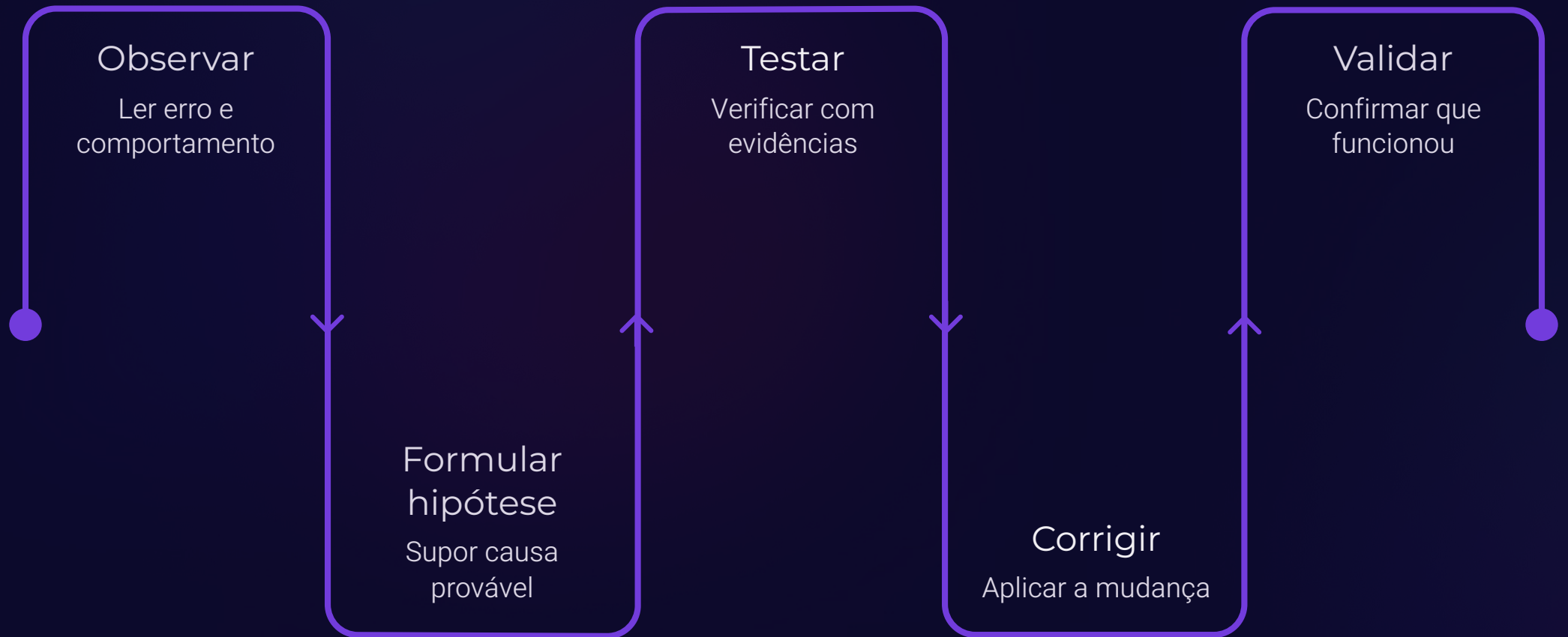
Investigando antes de corrigir

Depurar não é adivinhar. É seguir pistas, testar hipóteses e validar soluções.



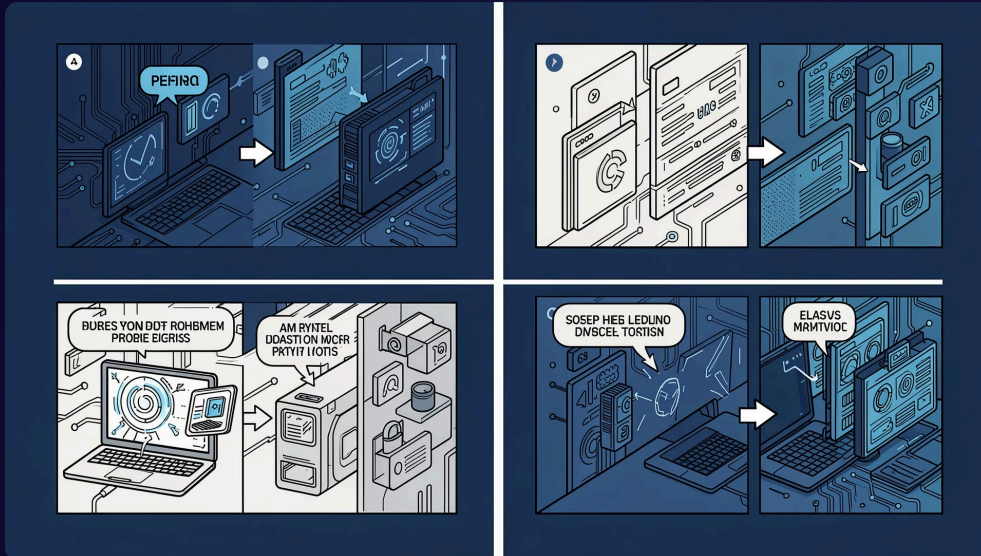
Debugging não é tentativa e erro

Corrigir bugs sem método pode gerar mais confusão. **Antes de mudar o código, entenda o problema.**



Seguir esse fluxo transforma o debugging em investigação — não em chute.

Divide and Conquer



O que é?

Dividir o problema em partes menores para descobrir **onde exatamente a falha acontece**.

  Se o problema parece grande, divida em partes menores.

Fluxo do programa



Como aplicar Divide and Conquer?

1

Identifique o erro

Onde o comportamento inesperado aparece?

2

Separe em blocos

Divida o código em trechos menores e independentes.

3

Teste uma parte por vez

Isolando blocos, você descobre qual está errado.

4

Encontre o trecho culpado

O comportamento muda? O erro está aqui.

```
def calcular_media(notas):  
    soma = sum(notas) # Bloco 1 ✓  
    media = soma / len(notas) # Bloco 2 ← erro aqui?  
    return media
```


Print Statements

Como funciona?

Inserir `print()` temporários para **observar valores e caminhos** do programa durante a execução.



Prints funcionam como lanternas dentro do código.

Exemplo em Python

```
x = 5
print("valor de x:", x)

if x > 3:
    print("entrou no if")
    y = x * 2
    print("y =", y)
```

Saída: valor de x: 5 → entrou no if → y = 10

Testes de Hipóteses

Criar uma suposição sobre a causa do erro e testá-la de forma controlada, **sem mudar o código aleatoriamente.**

🤔 Hipótese

"O erro acontece porque a variável está vazia."

🧪 Teste

"Vou imprimir o valor antes do cálculo."

✅ Resultado

A hipótese foi confirmada ou descartada.

📌 Uma boa hipótese evita mudanças aleatórias no código.

Tracing - Execução passo a passo

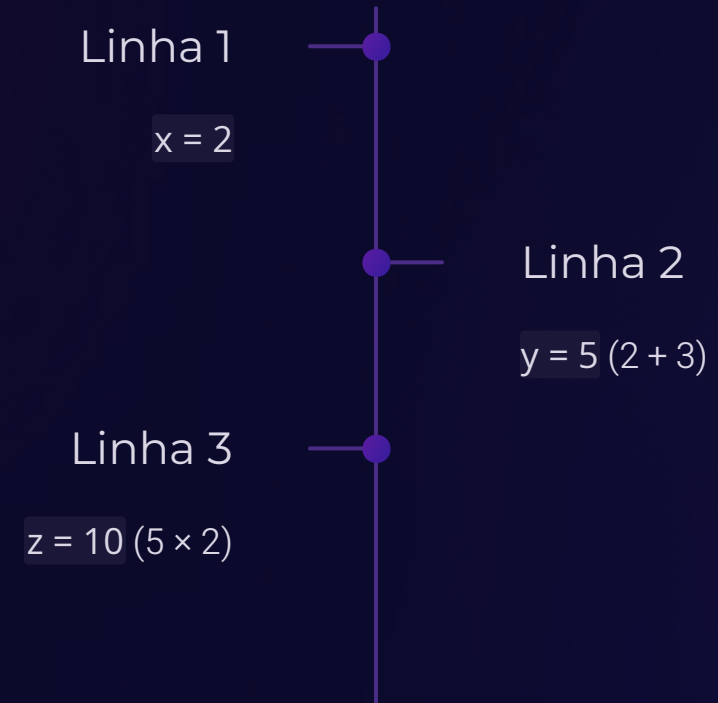
O que é?

Acompanhar o código **linha por linha**, registrando como os valores das variáveis mudam a cada passo.

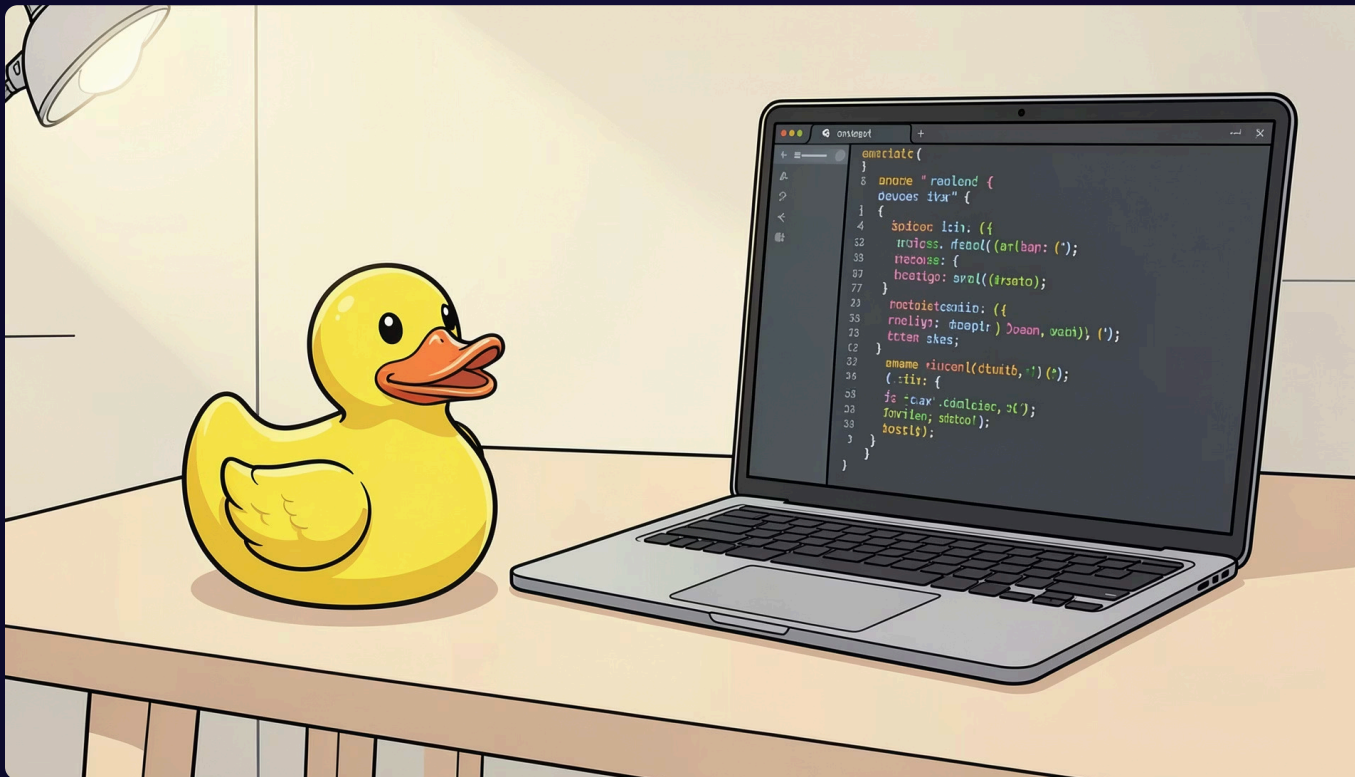
🔍 Às vezes o bug não está no resultado final, mas no caminho.

Exemplo: rastreando valores

```
x = 2  
y = x + 3  
z = y * 2
```



Rubber Duck Debugging



Como funciona?

Explicar o código **em voz alta**, para um colega, um objeto ou até para você mesmo, ajuda a perceber falhas que estavam passando despercebidas.

"Esta linha deveria calcular a média... mas por que a divisão acontece *antes* da soma?"

  Se você consegue explicar, consegue investigar melhor.



ESTRATÉGIA 6

Code Review Colaborativo

Revisar o código com outra pessoa ajuda a encontrar erros e validar hipóteses com mais rapidez.



Piloto

Manipula o código, digita e executa os testes.



Navegador

Observa, questiona e sugere caminhos sem tocar no teclado.



Explicar o código para alguém ajuda a enxergar o erro com mais clareza.

Qual estratégia usar?

Cada situação pede uma abordagem diferente. Use este guia rápido:

Situação	Estratégia
Problema grande ou confuso	Divide and Conquer
Não sei o valor das variáveis	Print Statements
Tenho uma suspeita sobre a causa	Testes de Hipóteses
Preciso entender o caminho do programa	Tracing
Não consigo enxergar o erro sozinho	Rubber Duck
Preciso validar minha solução	Code Review Colaborativo